

Choosing a cache policy from a short request prefix

Gennaro Francesco Landi · 30 September 2026 · Research artifact, version 1.0

A seven-trace experiment shows a small selector can recover most of the measured oracle's human hit rate. Its useful result comes with a narrow workload boundary.

59 of 63 settings, 93.7% correct family selection.

A tree trained on six traces predicts the nine settings of the seventh: three overlay behaviors and three target agent fractions. Across all seven folds, its mean human hit rate is 13.09%, versus 12.94% for the strongest fixed guarded policy.

The systems question

A cache has limited space. A request stream with repeated objects rewards different choices from a stream of largely one-off accesses, and object sizes change the cost of admitting the wrong object. The question here is whether a short prefix can help choose among WTinyLFU, S3FIFO, and GDSF before committing to one family.

The study combines bounded windows from seven public Meta, Wikimedia, and Tencent traces with a **WebLINX-derived synthetic agent overlay**. It fixes cache capacity at 64 MiB to expose policy differences under pressure. The overlay is a controlled workload proxy; it does not establish behavior under production AI-agent traffic.

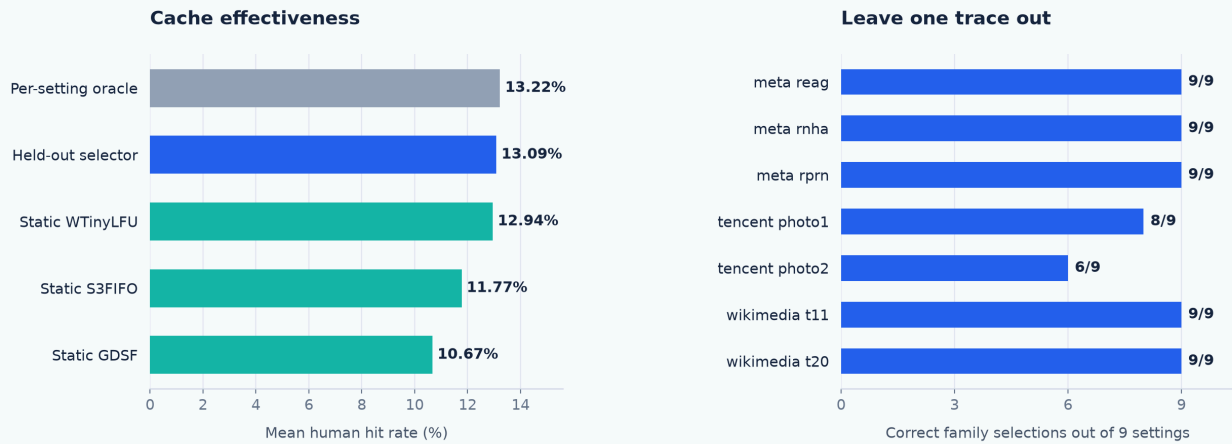
The experiment

Seven traces, three overlay behaviors, and three target agent fractions give 63 settings. Each setting supplies statistics from a 1,000-request mixed prefix. The fitter considers four statistics and chooses a tree using at most two. Every evaluation fold holds out an entire trace and trains on the remaining six.

Archived measurements contain 513 policy cells. The Meta and Wikimedia panels compare nine policy variants per setting; the Tencent panels compare six. The selector deploys a rate-guarded representative of its chosen family. The oracle is the best measured variant per setting, including other guard variants where available.

CacheRegime

59 / 63 correct family choices · 7 public traces · 64 MIB cache



Controlled WebLINX-derived synthetic agent overlay; three behaviors × three agent fractions per trace.
Human hit rates are means across 63 settings. Selector and static baselines use rate guards; oracle may choose any measured variant.
Source: results/summary.json · Refit from frozen features; full raw-trace simulation is outside this artifact.

Figure 1. Archived V1 simulation panel with a synthetic overlay. Hit-rate bars are unweighted means over 63 settings; right-hand counts show every held-out trace. Source: recomputed metrics.

What the result supports

The selector's mean human hit rate is 0.1396 percentage points below the measured oracle. Its gain over the strongest fixed baseline is 0.1409 percentage points. That aggregate improvement is modest. The more useful finding is that the prefix can identify the measured winning family in 59 settings, and the artifact makes the four errors visible rather than hiding them behind an average.

All four errors occur on the two Tencent traces, with 8/9 and 6/9 correct family predictions. The other five traces each have 9/9. These are seven trace groups with correlated within-trace settings, so 63 must not be read as the number of independent deployments.

What remains unresolved

The window size, feature menu, and candidate families were developed on the same research panel. Whole-trace holdout refits thresholds without the test trace, but does not provide a separate untouched dataset for choosing the research procedure. The source panels also differ in policy coverage and simulator warmup.

A follow-up using collected browser-mediated agent traffic produced winners outside the original family space. Its diagnostic uses different feature extraction and workload construction, so it cannot isolate the effect of changing the overlay. It limits any claim that the original selector or taxonomy transfers automatically. The public artifact includes the restricted-subset counts and qualifications, with no claim of production validation.

An inspectable result

`python3 scripts/reproduce.py` verifies all input hashes, reconstructs winners and baseline rates from the 513 archived cells, refits the seven held-out folds, and checks every prediction against the archived record. The numerical reproduction requires only Python's standard library.

The release starts from frozen prefix features and aggregate simulation measurements. Reconstructing those features or rerunning the original simulator requires separate raw inputs and environment work. Source hashes, methods, and every setting-level result are available alongside the figure.

Artifact overview · Full methods · Provenance · Source attribution · Presentation